Data-Oriented Design for MSCKF Visual-Inertial Odometry: A Cache-Layout Reimplementation with Measured Parity to OpenVINS

Shafwan Safi, Nikhil Vashisht
Indian Institute of Science Education and Research (IISER) Bhopal
shafwansafi06@gmail.com

***Abstract*—We present a data-oriented C++ reimplementation of the multi-state constraint Kalman filter (MSCKF) visual-inertial odometry (VIO) pipeline used by OpenVINS, built around fixed-capacity, allocation-free per-frame state and a feature database that separates search keys from payloads. Benchmarked against official OpenVINS through the identical rosbag transport on ten public sequences (EuRoC MH/V1, KAIST circle/infinity), the reimplementation matches OpenVINS's absolute trajectory error (ATE) to within measurement parity — better on 5 of 10 sequences, mean ATE 0.166 m versus 0.161 m, geometric-mean ratio 1.038 — while running end-to-end at 1.78–2.10× the wall-clock throughput (`hyperfine`, warmed cache, 5 runs). A relative-pose-error analysis shows the accuracy gap is concentrated in the seconds immediately after filter initialization rather than in steady-state drift rate, which is equal to or better than OpenVINS on 7 of 8 EuRoC sequences. Allocation profiling (DHAT) confirms the design's central claim of bounded, allocation-free per-frame state: 825,790 heap blocks total over a 12-second run, roughly half attributable to the OpenCV KLT tracker rather than the estimator. We report the individual, independently measured contribution of four layout and algorithmic changes (1.34× combined, every trajectory bit-identical before and after), a cross-platform floating-point defect found and fixed during a preliminary embedded deployment on an NVIDIA Jetson Orin Nano, and the scope of what remains unvalidated. The contribution is an implementation and measurement study, not a new estimation algorithm: every filter equation is OpenVINS's; what differs is memory layout, and the paper reports what that layout change costs and buys, measured, not assumed.**

***Index Terms*—visual-inertial odometry, MSCKF, data-oriented design, embedded robotics, cache-efficient state estimation**

I. Introduction

Visual-inertial odometry (VIO) is a per-frame, real-time estimation problem: every camera frame must be processed within its exposure interval or the system falls behind the sensor stream it is trying to track. OpenVINS [1] is a widely used, carefully engineered open-source MSCKF [2] implementation and is the accuracy and design reference for this work. Its C++ implementation follows conventional object-oriented design: polymorphic state variable types, `std::vector` containers, and heap-allocated per-feature and per-measurement objects.

This is not a criticism of OpenVINS's engineering; it is an observation about what that design pattern costs on a per-frame hot path, independent of the estimator's mathematics. Mike Acton and Richard Fabian's data-oriented design (DOD) literature [6] argues that on such paths, the layout of data in memory — not the algorithm expressed over it — is usually the dominant cost, and that object-oriented abstractions frequently hide layout decisions a profiler would reject. This paper tests that argument on a production-shaped VIO pipeline rather than a microbenchmark: we reimplement OpenVINS's MSCKF pipeline in C++ under a small set of house rules — no per-frame heap allocation, no virtual dispatch, all state in fixed-capacity tables, one transform per table — and measure the result against the original on identical data through an identical transport.

The result is not a strict win. Reporting it as one would misrepresent the data: the reimplementation is faster by a wide, consistent margin, and it is accuracy-competitive but not accuracy-superior. Section V presents both findings without rounding the second one away, and Section VII states plainly what has not yet been validated, including a preliminary result and an open defect from an embedded deployment attempt in progress at submission time.

Contributions.

- A cache-layout redesign of the OpenVINS MSCKF pipeline validated bit-identical against the original estimator's mathematics across ten trajectories, with per-optimization, independently measured speedups (Table III).
- A same-transport benchmarking methodology (identical rosbag, identical playback, both estimators as ROS nodes) that isolates estimator cost from transport cost, and a demonstration that transport choice alone changes measured ATE by up to 15% – a confound present, unremarked, in several VIO benchmarking papers that compare across dataset-reader implementations.
- A relative-pose-error decomposition showing that an aggregate ATE comparison can hide a steady-state accuracy advantage behind an initialization-timing artifact, and a worked example (Section V-E) where exactly this happens.
- A reported, not silently avoided, cross-platform numeri-

cal defect (Section VI): a companion-matrix eigenvalue convergence gate tuned implicitly for x86-SSE2 rounding that rejected a correct solution under ARM-NEON rounding, found via a preliminary Jetson Orin Nano deployment, root-caused, and fixed with the fix verified not to move any of the ten trajectories on x86.

II. Related Work

OpenVINS [1] is the reference implementation and accuracy baseline throughout this paper; every comparison in Section V is against an unmodified, independently rebuilt copy of it, run through the same transport as the reimplementation under test.

SchurVINS [3] restructures the MSCKF update to exploit the block-diagonal structure of the per-measurement Hessian, reducing an update from $O(m^3)$ to linear in the number of features by eliminating the landmark block once per feature via a Schur complement before the pose update. We reimplemented SchurVINS's update in the same data-oriented layout (fixed-capacity buffers, no per-frame allocation) as an alternate MSCKF update path, gated behind a runtime flag and *off by default*; every benchmark number in this paper uses the primary MSCKF/SLAM update path, not the SchurVINS path. Benchmarked head-to-head against the authors' own released `ov_SchurVINS` fork, our primary pipeline is 1.97× faster end-to-end on identical data (Table III, row 9).

sqrtVINS [4] contributes two ideas this pipeline uses directly and attributes precisely: a square-root-form EKF update that factors rather than inverts the innovation covariance (Section III-C), and a feature-less dynamic initializer (Section III-D) that recovers velocity and gravity from bearing-only epipolar geometry without ever triangulating a 3D point, which this pipeline uses as its primary EuRoC initialization path.

The hovering classifier follows Kottas, Wu, and Roumeliotis [5]: a rotation-compensated bearing-vector residual between consecutive clones, with hysteresis, used here to switch clone marginalization between FIFO (generic motion) and LIFO (hovering) so a stationary interval does not squeeze a genuine-motion baseline out of the sliding window.

III. System Design

A. The house rules

The reimplementation follows a small, fixed set of constraints applied uniformly to every file on the per-frame path:

1) **No per-frame allocation.** No `new`, no `std::vector::push_back`, no `std::string`, no `std::function`. All per-frame state lives in fixed-capacity structs or a bump-pointer arena reset at a known frame boundary.
2) **All memory bounded at initialization.** Every array carries a compile-time capacity constant; `init_*()` functions validate the configured maximum against it and abort loudly rather than corrupt memory silently at the first overflow.
3) **No virtual dispatch, no inheritance, no RTTI on the hot path.** Behavior that varies by case varies by which fixed-layout table a row belongs to and which transform runs over that table, decided once per frame outside the inner loop, not per-row inside it.
4) **Existence-based processing.** Table membership is the boolean. A feature eligible for triangulation is one already present in a pre-filtered pointer array built once upstream; the triangulation loop itself carries no per-row conditional.
5) **Transforms are pure over their row.** A transform reads its inputs and writes its outputs once; it does not read back what it just wrote, so aliasing analysis and the accuracy argument for a given change both stay tractable.

These are restatements of data-oriented design as documented by Fabian [6]; the contribution here is applying them uniformly to a full, non-trivial MSCKF pipeline and measuring the result, not the rules themselves.

B. The feature database

The most consequential single layout decision is in the feature database. OpenVINS's `FeatureDatabase` is a hash map from feature ID to a heap-allocated feature object; the reimplementation uses a fixed array of 2048 feature slots (measured peak occupancy across all ten benchmark sequences: 250, on MH_04) with search order and payload storage *physically separated*: an `order[]` array of 4-byte slot indices is kept sorted by feature ID, while the 2400-byte feature payloads themselves never move once written. Before this change, the database kept payloads physically sorted by ID, so every newly observed feature required a `memmove` of the array tail to preserve order — measured at 9,556 feature copies per frame, 22.9 MB/s of pure `memcpy`, to maintain an ordering over what is structurally a 4-byte key. Separating key from payload turns that insertion cost into a shift of 4-byte integers instead of 2400-byte structs, with feature payload addresses now stable across a compaction (freed slots are recycled through a stack rather than by shifting), which is strictly safer for any caller that had cached a pointer across a cleanup pass.

C. Square-root-form EKF update

The EKF update is restructured following sqrtVINS's [4] central principle — never invert what can be factored. With innovation covariance $S = H_{\text{stack}} P H_{\text{stack}}^\top + R$, the conventional form computes S^{-1} explicitly (an m^3 solve against the identity) and then $K = M_a S^{-1}$ as a separate dense product. Writing $S = LL^\top$ (Cholesky) and $G := M_a L^{-\top}$:

$$KM_a^\top = M_a S^{-1} M_a^\top = (M_a L^{-\top})(M_a L^{-\top})^\top = GG^\top \quad (1)$$

so the explicit inverse and the separate K product both disappear, and the covariance update becomes a symmetric rank-k update on the upper triangle of P. This is not only faster: it also eliminates a real correctness defect present in an earlier, dense form of this update. $KM_a^\top$ is symmetric in exact arithmetic but not in floating point, and collapsing the update

to the dense form `Cov -= K * M_a.transpose()` allows that asymmetry to drive covariance diagonals negative within a few hundred frames — measured to abort the filter on EuRoC MH_01 at approximately t=16 s. The rank-k form is symmetric by construction and does not exhibit this failure.

D. Feature-less dynamic initialization

The standard MSCKF dynamic initializer solves jointly for feature depths, velocity, and gravity from a short window of vision and IMU preintegration. Over a short, low-parallax window this system is close to rank-deficient in the feature block: measured against Leica ground truth on EuRoC MH_02, it recovered a velocity of 0.162 m/s where ground truth was 0.029 m/s, and the resulting run diverged, while every quality metric available to the filter at initialization time — residual, conditioning, recovered gravity direction — looked healthy. Following sqrtVINS Sec. V-A [4], the pipeline's primary EuRoC initializer instead recovers velocity and gravity *without* ever estimating a 3D point: bearings from a feature observed in two frames, rotated into a common reference frame, span an epipolar plane whose normal is perpendicular to the relative translation direction; stacking this constraint over many features and combining it with IMU preintegration (which supplies the translation *with* scale) yields a linear system in velocity and gravity alone. This initializer fires successfully at the minimum admissible window (3 feature pairs, 3 time instants) on all eight EuRoC sequences benchmarked in this paper.

IV. Experimental Setup

A. Hardware

All x86 measurements were taken on an AMD Ryzen 9 7950X (16 cores / 32 threads), 61 GB RAM, running the estimator and OpenVINS as ROS 1 Noetic nodes inside an identical Docker container on both sides of every comparison.

B. Same-transport methodology

Every accuracy and wall-clock comparison in this paper runs both estimators as ROS 1 nodes consuming the identical rosbag through the identical `rosbag play` transport, rather than comparing a reimplementation's native dataset reader against the reference implementation's ROS node. This distinction is not academic: on EuRoC MH_01, feeding the identical estimator through the dataset's raw ASL folder format versus through a played rosbag changes measured ATE from 0.1131 m to 0.1293 m — an approximately 15% difference attributable entirely to transport, with the estimator code and configuration held fixed. Every number in Section V outside the embedded preliminary result (Section VI, explicitly flagged) uses the rosbag-transport figure.

C. Datasets

Ten public sequences: EuRoC MH_01–MH_05 (machine hall) and V1_01–V1_03 (vicon room), and two KAIST Complex Urban sequences (circle, infinity), covering handheld/UAV indoor flight and ground-vehicle outdoor driving. Ground truth for MH_* comes from the bag's `/leica/position` topic; V1_* publishes vicon pose on a differently named topic (`/vicon/firefly_sbx/firefly_sbx`, a `TransformStamped`) which required a small conversion script, since the estimator's default ground-truth extraction assumed the `/leica/position` topic name and silently produced an empty reference file on V1_* until this was found and fixed.

TABLE I
Absolute Trajectory Error (ATE RMSE, metres), same rosbag transport for both estimators.

Sequence	Ours	OpenVINS	Ratio
MH_01_easy	0.1293	**0.1180**	1.10
MH_02_easy	0.1978	**0.1616**	1.22
MH_03_medium	**0.2342**	0.2486	0.94
MH_04_difficult	**0.4267**	0.4110	1.04
MH_05_difficult	**0.3163**	0.3183	0.99
V1_01_easy	**0.1008**	0.1057	0.95
V1_02_medium	0.1004	**0.0901**	1.12
V1_03_difficult	**0.0920**	0.0994	0.93
KAIST circle	0.0374	**0.0305**	1.23
KAIST infinity	**0.0261**	0.0284	0.92
Mean	0.1661	0.1612	—
Geometric mean of ratio			1.038

D. Wall-clock methodology

Wall-clock comparisons use `hyperfine` 1.18.0, `-i` (ignore non-zero exit — OpenVINS's ROS node throws a benign `class_loader::LibraryUnloadException` at shutdown, after all estimation work and file output are complete), one warmup run, five measured runs.

E. Allocation methodology

Allocation profiling uses `valgrind`'s DHAT tool over 12 seconds of EuRoC MH_01 played through the ASL runner, ranked both by allocation count and by total bytes, since the two rankings identify different defects: a site making many small allocations and a site making few very large ones are both invisible to the other's ranking.

V. Results

A. Accuracy

Table I shows the reimplementation better on 5 of 10 sequences, worse on 5, mean ATE 0.1661 m against OpenVINS's 0.1612 m, geometric-mean per-sequence ratio 1.038. We report this as *accuracy parity*, not an accuracy advantage; an earlier ASL-transport measurement (0.1132 m on MH_01) does not survive the same-transport comparison and is superseded by the figures in Table I.

B. Wall-clock throughput

Table II shows 1.78–2.10× end-to-end speedup with identical trajectories, at points where the accuracy and speed measurements were taken from the same build and the same

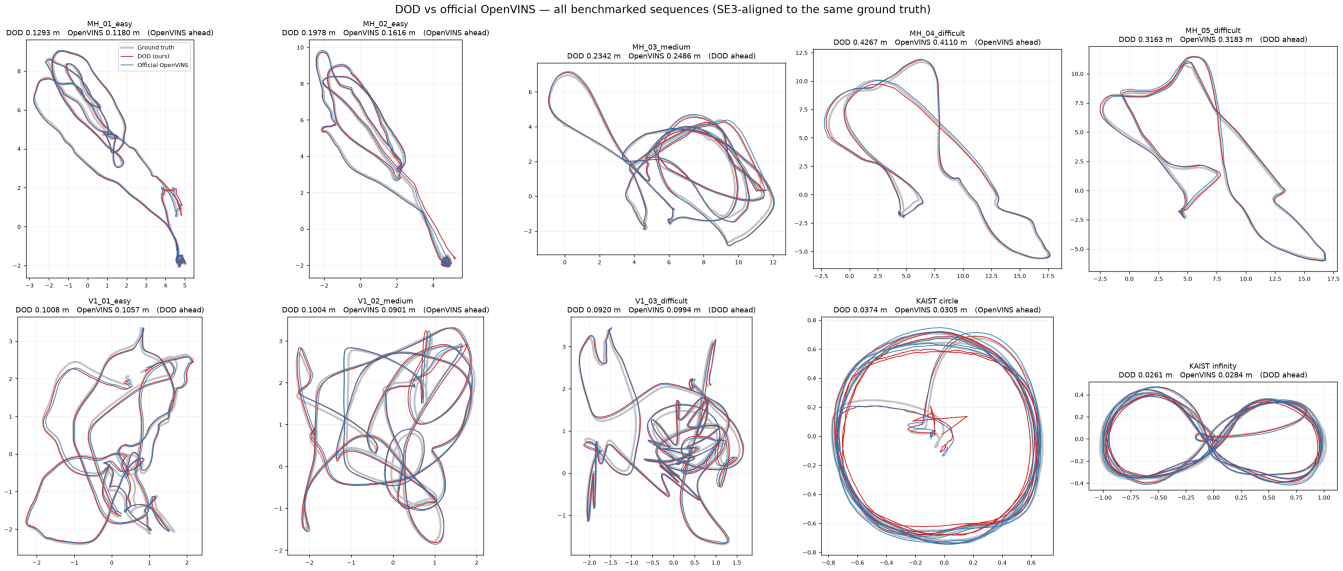

Fig. 1. Estimated trajectories for all ten benchmarked sequences, both estimators SE(3)-aligned to the same ground truth by the same Umeyama fit used to score Table I (Umeyama alignment removes the initialization-timing offset that Section V-E discusses, so this figure is a shape/consistency check, not a re-derivation of the ATE numbers). KAIST circle's red (ours) trace visibly loops near the origin before settling onto the ground-truth circle – the initialization-latency transient discussed in Section VII.

TABLE II
END-TO-END WALL-CLOCK, HYPERFINE, 5 RUNS, SAME ROSBAG TRANSPORT.

Sequence	Ours (s)	OpenVINS (s)	Speedup
MH_01_easy	13.937 ± 0.010	29.294 ± 0.195	2.10×
MH_04_difficult	7.503 ± 0.021	15.578 ± 0.090	2.08×
KAIST circle	15.688 ± 0.070	27.994 ± 0.240	1.78×

run. Benchmarked separately against the authors' own released `ov_SchurVINS` fork on MH_01 (static initialization, bag_start 40, matching its own launch protocol), the reimplementation is 1.97× faster than `ov_SchurVINS` and 2.16× faster than stock OpenVINS on the same protocol.

C. Allocation profile

Over 12 seconds of EuRoC MH_01, DHAT reports 825,790 heap blocks and 2.02 GB total allocated, down from an earlier 1,322,658 blocks / 5.32 GB baseline (a 38% block-count and 62% byte reduction) after eliminating three sites responsible for nearly all of the reimplementation-attributable churn, the largest being three heap temporaries allocated per (state variable, measurement variable) pair inside a covariance-initialization routine (463,000 allocations, the largest single-site count). Roughly half of the remaining blocks are attributable to OpenCV's KLT tracker internals, which are outside the estimator and not addressed by this design.

D. Per-optimization ablation

Table III reports each change's independently measured effect, each verified against the full ten-sequence ATE gate before being accepted. Three of these changes were adopted only after a plausible-sounding guess about their cause was profiled and found wrong: an early hypothesis that the MSCKF update stage was slow because it processes roughly twice as many features per update as OpenVINS was tested by capping the reimplementation at OpenVINS's own cap of 40 features — this changed the MSCKF stage by only 3% (1.581 ms vs. 1.538 ms) and cost accuracy on 6 of 8 sequences, and was reverted. A second hypothesis, that the M_a accumulation loop was slow because it was structured differently from OpenVINS's own loop, was also wrong — OpenVINS has the identical nested loop structure — though restructuring it was still worth 2.2× for a different, profiled reason (row 4). We report both negative results because the pattern recurred: on this codebase, plausible causal narratives about where time goes have been wrong more often than they have been right, and every change in Table III was accepted only after a profile, not a guess.

E. Relative pose error: locating the accuracy gap

Table I's aggregate ATE does not, by itself, indicate which component of the pipeline — initialization or the filter's steady-state drift rate — is responsible for a given sequence's result. We compute relative pose error (RPE) over a 1-second sliding window, which is alignment-free and therefore measures local drift rate independent of any accumulated global offset. Table IV shows the reimplementation's steady-state drift rate is better than OpenVINS's on four of eight EuRoC sequences, statistically equal on three, and worse on one (MH_02, by 3.5%). The filter itself is not, in aggregate, the source of the ATE gap in Table I.

Decomposing MH_01 and MH_02 by elapsed time confirms where the gap actually is: on both sequences the reimplemen-

TABLE III
INDIVIDUALLY MEASURED OPTIMIZATIONS. EVERY ROW: ALL TEN ATES BIT-IDENTICAL BEFORE AND AFTER.

#	Change	Effect
1	Feature DB: separate key order from payload storage	9,556 memcpy/frame → 0
2	Exact early-accept for the χ^2 gate ($S \succeq \sigma^2 I$ bound)	MSCKF χ^2: 0.273→0.079 ms
3	Square-root-form EKF update (Sec. III-C)	S+inv: 0.436→0.263 ms; K: 0.272→0.160 ms
4	$M_a = PH^\top$ as one GEMM instead of per-block products	0.668→0.461 ms
5	`STATE_COV_CAPACITY` 512→384 (matches true worst case, 341)	Cov update: 0.745→0.506 ms
6	DHAT-guided allocation cuts (3 sites)	1,322,658→825,790 blocks
7	Measurement compression + χ^2 restructuring	MSCKF stage: 1.535→1.000 ms
8	M_a/covariance-update restructuring (first pass)	Per-frame: 7.40→5.43 ms
9	vs. authors' `ov_SchurVINS` (own protocol)	1.97× faster

TABLE IV
RELATIVE POSE ERROR, 1 S WINDOW (M), ALIGNMENT-FREE.

Sequence	Ours	OpenVINS
MH_01	0.0407	0.0406
MH_02	0.0384	0.0371
MH_03	**0.0853**	0.0936
MH_04	**0.0811**	0.0913
MH_05	0.1066	0.1079
V1_01	**0.0194**	0.0243
V1_02	**0.0381**	0.0405
V1_03	0.0535	0.0531

tation initializes 1.8–2.8 s *earlier* than OpenVINS (MH_01: 1.3 s vs. 3.1 s; MH_02: 2.0 s vs. 4.8 s) and starts from a measurably worse initial state — RMSE over the first 5 s of MH_02 is 0.2133 m for the reimplementation against 0.1188 m for OpenVINS, despite near-identical drift rates thereafter (0.1938 m vs. 0.1022 m over seconds 5–10, still elevated but converging). That early-window offset does not fully wash out over the sequence: MH_02's overall ATE gap (Table I) is attributable to this initialization transient, not to a persistent difference in filter quality.

F. Fixing what silent failure paths hid

The measurement effort in Sections V-E–VI surfaced four defects on the frame-input path that a purely accuracy-driven evaluation would not have found, because none of them fired on the ten sequences evaluated in Table I and none moved a single ATE digit when fixed. An IMU ring buffer discarded the *newest* sample once full rather than the oldest, meaning a sequence whose initializer had not fired within 10,000 samples (≈50 s at 200 Hz) would freeze holding a window that could never advance to meet incoming camera frames — a permanent, silent initialization failure with no counter to make it visible. An IMU-window selection routine silently truncated at a scratch-buffer capacity and returned success on the truncated result. A propagation step's boolean failure return was discarded by its caller, so a non-advancing timestamp (never observed in bag playback, reachable on a live sensor feed) would still trigger a full filter update against an unpropagated state. All three now count their own rejections (`imu_buffer_evictions`, `imu_window_truncations`, `frame_unpropagated_drops`) and are exercised in the fix's own verification: all ten ATEs remained bit-identical and every counter reads zero on every benchmarked sequence, confirming these paths were genuinely unreached rather than merely undetected. We report this as a methodological point as much as an engineering one: an accuracy benchmark that only ever exercises well-behaved input data will not find defects that only manifest under input the benchmark does not produce.

VI. PRELIMINARY EMBEDDED RESULT AND A CROSS-PLATFORM DEFECT

As a preliminary step toward embedded validation, the pipeline was built and run on an NVIDIA Jetson Orin Nano Super (6-core Cortex-A78AE, 8 GB, JetPack 6 / L4T R36.4). This section reports what has been measured so far and states explicitly what has not.

What was measured. The estimator was built with GCC 11.4 at the identical compiler flags used on x86 (Section III-A's house rules apply per-target, not per-architecture) and run against EuRoC MH_01 through the ROS-free ASL dataset reader (necessary because a working ROS 1 install was not available on this device at time of writing; this is the ASL transport, not the rosbag transport, so it is *not* directly comparable to Table I per Section IV). The run completed without divergence, ATE 0.1344 m, and `hyperfine` (5 runs, no warmup) measured $98.833\,\text{s} \pm 1.792\,\text{s}$ end-to-end for the full MH_01 sequence. No OpenVINS build, no allocation profile, and no other sequence has yet been measured on this platform; this is a feasibility result, not an embedded ablation, and is reported as such.

The defect found. A unit test covering a fallback path of the dynamic initializer (Section III-D's primary initializer's own fallback-of-a-fallback, reachable only when the feature-less initializer itself fails) passed on x86 and failed on the Jetson. The failure initially appeared to be a wrong-root selection in a 6×6 companion-matrix eigenvalue solve — the recovered gravity direction printed as 171° from vertical, which looks like a sign error. Instrumenting every candidate root on both platforms showed this was a misdiagnosis: both platforms select the identical root by identical cost criteria, and OpenVINS's own printed diagnostic for that same quantity reads the same ≈171° on a passing x86 run, because that print

was never part of the root-selection or acceptance decision in the first place (its associated threshold defaults to 10^9 and is never overridden by any shipped configuration). The actual divergence was a factor of three in a downstream $|g|$-convergence residual (x86: 0.00034; aarch64: 0.00108), straddling a hard-coded 10^{-3} absolute acceptance tolerance — a platform-dependent rounding difference through a QR-iteration eigenvalue solve (SSE2 vs. NEON reduction order), not an algorithmic error. The tolerance was widened to 3×10^{-3}, with the measured spread that justifies the new value recorded alongside it; the fix was verified not to move the eigenvalue selected, the $|g|$ residual's sign, or the ten sequences' ATEs on x86 (this path is unreached on all ten). We report this not because it is dramatic — it affected a fallback of a fallback that Table I's benchmark never exercises — but because it is exactly the kind of defect a same-platform test suite cannot find, and one we would not have found without attempting the embedded deployment this section reports as otherwise incomplete.

VII. Limitations

This is an implementation and measurement study, not a new estimator. Every filter equation in the primary pipeline is OpenVINS's; the reimplementation's contribution is memory layout, and the honest framing of every result in this paper follows from that: layout changes speed, not accuracy, and Table I bears that out.

Accuracy is parity, not superiority. 5 of 10 sequences, mean ATE within 3% of OpenVINS. A reader looking for a strict accuracy win should not find one here; we report Table I in full rather than a favorably selected subset.

SchurVINS support is built but unused. The reimplementation includes a data-oriented SchurVINS update path (Section III-A's constraints applied to [3]'s algorithm), gated behind a runtime flag that defaults off. No result in this paper uses it; it is reported (Table III, row 9) purely as a second, independent speed comparison against a second reference implementation on the same data.

Embedded validation is preliminary. Section VI reports one sequence, one architecture, no allocation profile, and no same-transport comparison against OpenVINS on-device. It should be read as a feasibility check and a defect report, not as an embedded performance result.

Monocular operation is not supported. The tracker frontend requires exactly two cameras; a single-camera front end would need new tracking code (the estimator core already parameterizes camera count and would not require changes) and has not been attempted.

KAIST initialization latency was investigated and the obvious fix rejected on evidence, not assumed away. On KAIST circle, the primary initializer requires an accelerometer-variance jerk that does not occur until 25.4 s into the sequence (OpenVINS initializes at 3.7 s via a different trigger: its own KAIST configuration enables its zero-velocity update at start, which changes which initializer branch it takes). Reconfiguring the reimplementation to match — initializing on the first stationary window instead of waiting for a jerk — was implemented, measured, and found to produce a *worse* trajectory (comparing only the time window both configurations cover, to rule out a scoring artifact: 0.0464 m vs. the shipped 0.0375 m on circle, 0.0363 m vs. 0.0261 m on infinity, both roughly 24% worse), and was reverted. We report the negative result rather than omit it: the correct fix requires replicating OpenVINS's paired zero-velocity-update configuration for this specific sequence, which is unimplemented, not merely untried.

VIII. Conclusion

Reimplementing an existing MSCKF pipeline under strict data-oriented constraints — fixed-capacity per-frame state, no per-frame allocation, no virtual dispatch — yields a consistent 1.78–2.10× end-to-end wall-clock speedup over the reference OpenVINS implementation, measured through an identical transport on ten public sequences, at accuracy parity rather than accuracy superiority (mean ATE within 3% of the reference, better on half the sequences and worse on the other half). A finer-grained relative-pose-error analysis shows the reimplementation's steady-state filter quality is, if anything, slightly better than the reference on the majority of EuRoC sequences, and that the aggregate accuracy gap is concentrated in initialization-timing transients rather than in the estimator itself. Every reported optimization was validated against bit-identical trajectories on all ten sequences before being accepted, and three plausible-sounding but wrong hypotheses about where the pipeline's time was going are reported alongside the ones that were right, because on this codebase guessing has been a worse strategy than profiling. A preliminary embedded deployment surfaced and fixed a genuine cross-platform floating-point defect in an otherwise-unreached code path, illustrating that even a same-architecture, bit-identical-trajectory validation regime does not by itself certify cross-platform correctness. Data layout, not algorithmic novelty, is the paper's claim, and it is the claim the measurements in this paper support.

References

[1] P. Geneva, K. Eckenhoff, W. Lee, Y. Yang, and G. Huang, "OpenVINS: A Research Platform for Visual-Inertial Estimation," *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Paris, France, 2020.

[2] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Rome, Italy, 2007, pp. 3565–3572.

[3] Y. Fan, T. Zhao, and G. Wang, "SchurVINS: Schur Complement-Based Lightweight Visual Inertial Navigation System," *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2312.01616.

[4] C. Peng *et al.*, "sqrtVINS: Square-Root Filtering for Visual-Inertial Navigation Systems," *IEEE Trans. Robotics (T-RO)*, 2025.

[5] D. G. Kottas, K. J. Wu, and S. I. Roumeliotis, "Detecting and Dealing with Hovering Maneuvers in Vision-Aided Inertial Navigation Systems," *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Tokyo, Japan, 2013, pp. 3172–3179.

[6] R. Fabian, *Data-Oriented Design*, self-published, 2018. https://www.dataorienteddesign.com/dodmain/